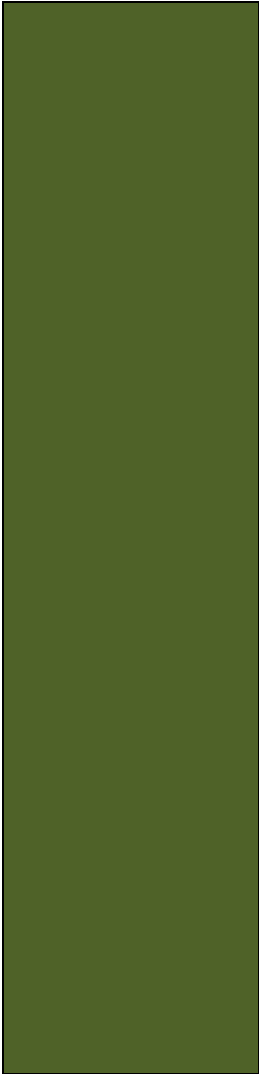


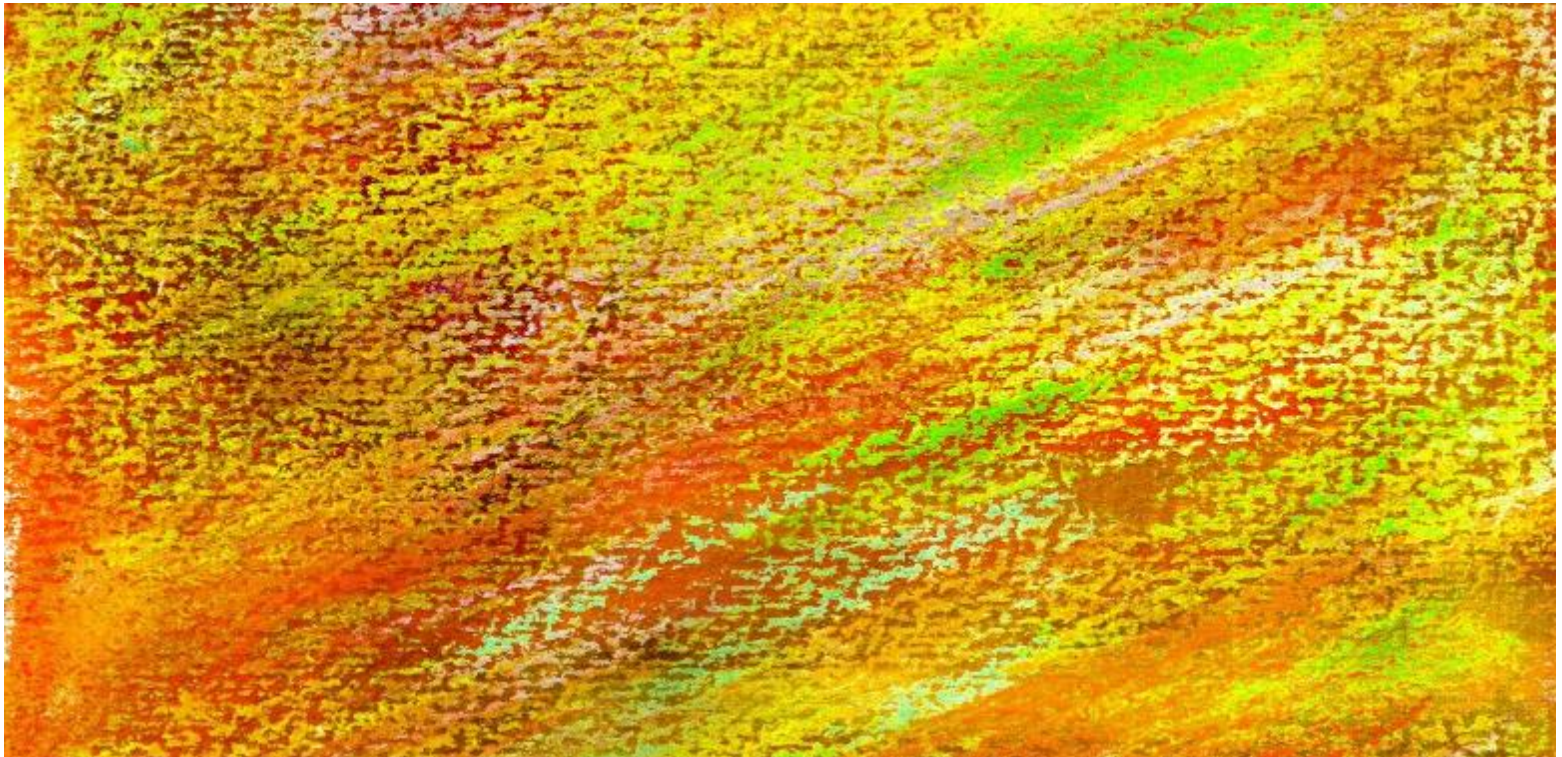


DAVID M. KROENKE and DAVID J. AUER
DATABASE CONCEPTS, 7th Edition



Chapter Five

Normalization





Chapter Objectives

- Learn how to transform E-R data models into relational designs
- Practice applying the normalization process
- Understand the need for denormalization
- Learn how to represent weak entities with the relational model
- Know how to represent 1:1, 1:N, and N:M binary relationships



Chapter Objectives (Cont'd)

- Know how to represent 1:1, 1:N, and N:M recursive relationships
- Learn SQL statements for creating joins over binary and recursive relationships



Transforming a Data Model into a Relational Design

1. Create a table for each entity:
 - Specify primary key (consider surrogate keys as appropriate)
 - Specify properties for each column:
 - Data type
 - Null status
 - Default value (if any)
 - Specify data constraints (if any)
 - Verify normalization
2. Create relationships by placing foreign keys:
 - Strong entity relationships (1:1, 1:N, N:M)
 - ID-dependent and non-ID-dependent weak entity relationships
 - Subtypes
 - Recursive (1:1, 1:N, N:M)



Representing Entities with the Relational Model

- Create a relation for each entity.
 - A relation has a descriptive name and a set of attributes that describe the entity.
- Specify a primary key.
- Specify column properties:
 - Data type
 - Null status
 - Default values (if any)
 - Data constraints (if any)
- The relation is then analyzed using the normalization rules.
- As normalization issues arise, the initial relation design may need to change.

Representing an Entity as a Table

ITEM

ItemNumber
Description
Cost
ListPrice
QuantityOnHand

(a) The ITEM Entity

ITEM

 ItemNumber
Description
Cost
ListPrice
QuantityOnHand

(b) The ITEM Table

ITEM (ItemNumber, Description, Cost, ListPrice, QuantityOnHand)

Figure 5-2: The ITEM Entity and Table



The Entity Table with Column Characteristics

ITEM (ItemNumber, Description, Cost, ListPrice, QuantityOnHand)

ITEM

 ItemNumber: int IDENTITY(10000,1)
Description: varchar(100) NOT NULL
Cost: numeric(9,2) NOT NULL
ListPrice: numeric(9,2) NULL
QuantityOnHand: int NOT NULL

Figure 5-3: The Final ITEM Table



Normalization Review: Modification Problems

- Tables that are not normalized will experience issues known as modification problems.
 - Insertion problems
 - Difficulties inserting data into a relation
 - Modification problems
 - Difficulties modifying data into a relation
 - Deletion problems
 - Difficulties deleting data from a relation



Normalization Review: Solving Modification Problems

- Most modification problems are solved by breaking an existing table into two or more tables through a process known as normalization.



Normalization Review: Definition Review

- Functional dependency
 - The relationship (within the relation) that describes how the value of one attribute may be used to find the value of another attribute.
- Determinant
 - The attribute that can be used to find the value of another attribute in the relation
 - The right-hand side of a functional dependency



Normalization Review: Definition Review II

- Candidate key
 - The value of a candidate key can be used to find the value of every other attribute in the table.
 - A simple candidate key consists of only one attribute.
 - A composite candidate key consists of more than one attribute.



Normalization Review: Normal Forms

- There are many defined normal forms:
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)
 - Boyce-Codd Normal Form (BCNF)
 - Fourth Normal Form (4NF)
 - Fifth Normal Form (5NF)
 - Domain/Key Normal Form (DK/NF)



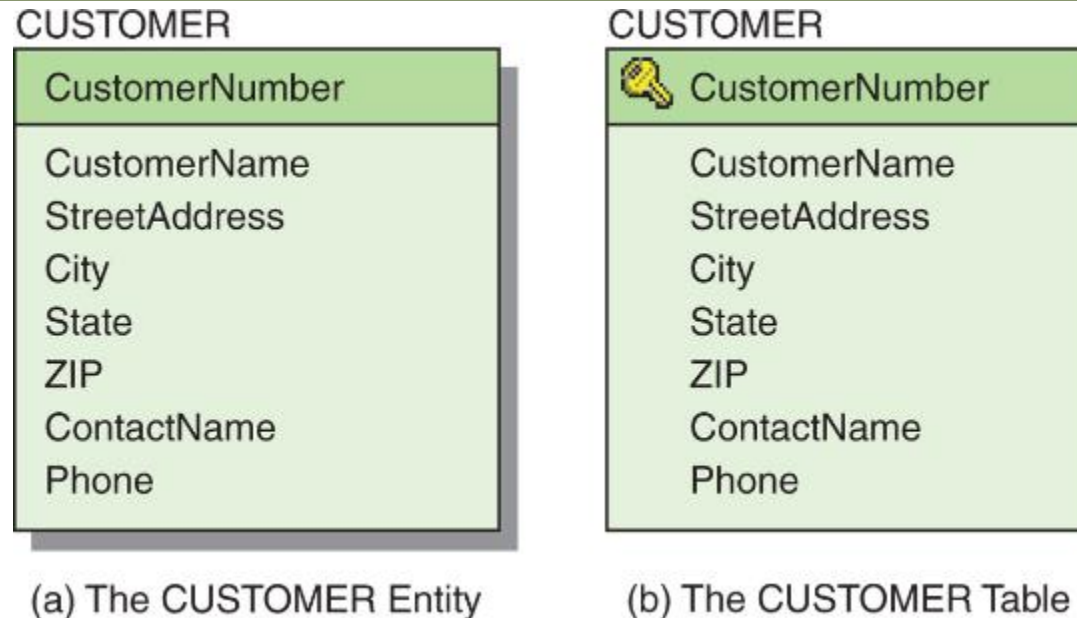
Normalization Review: Normalization

- For our purposes, a relation is considered normalized when:

Every determinant is a candidate key.

[Technically, this is Boyce-Codd Normal Form (BCNF)]

The CUSTOMER Table



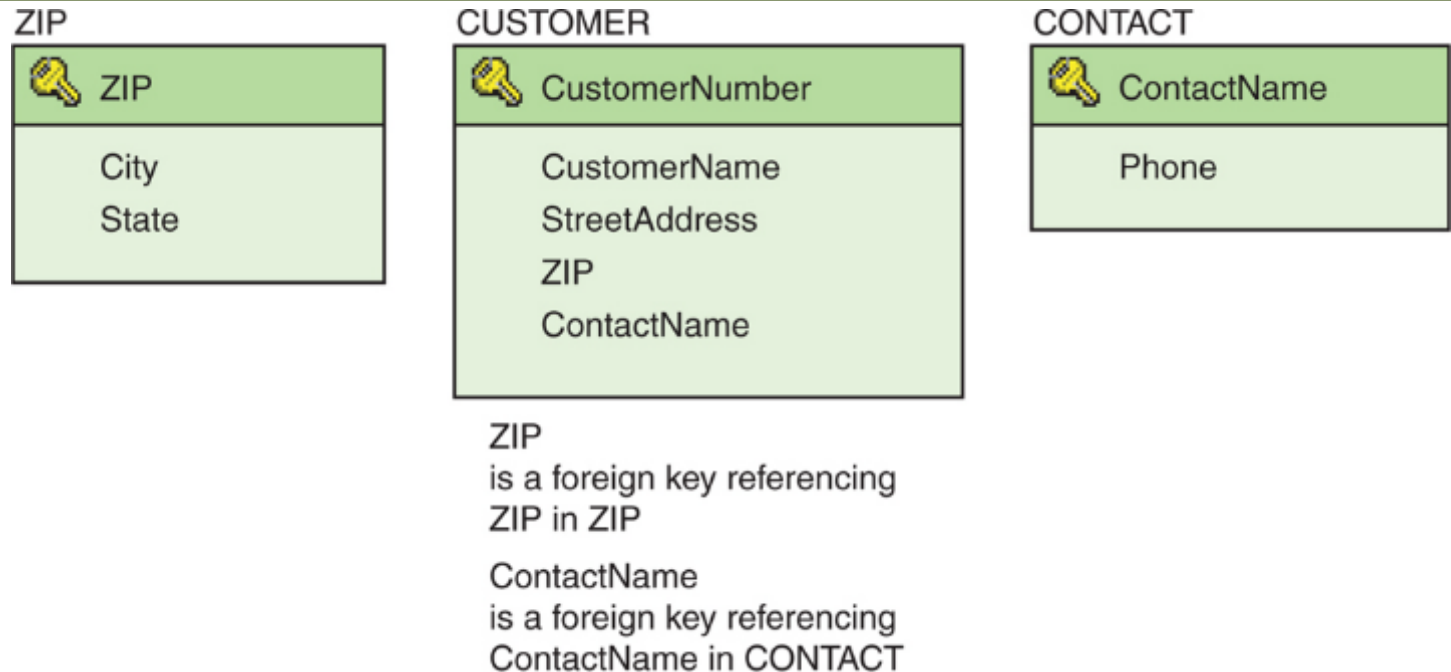
CUSTOMER (CustomerNumber, CustomerName, StreetAddress, City, State, ZIP, ContactName, Phone)

***ZIP* → (*City*, *State*)**

ContactName* → *Phone

Figure 5-4: The CUSTOMER Entity and Table

The CUSTOMER Entity: The Normalized Set of Tables



CUSTOMER (CustomerNumber, CustomerName, StreetAddress,
ZIP, ContactName)

ZIP (ZIP, City, State)

CONTACT (ContactName, Phone)

Figure 5-5: The Normalized CUSTOMER and Associated Tables

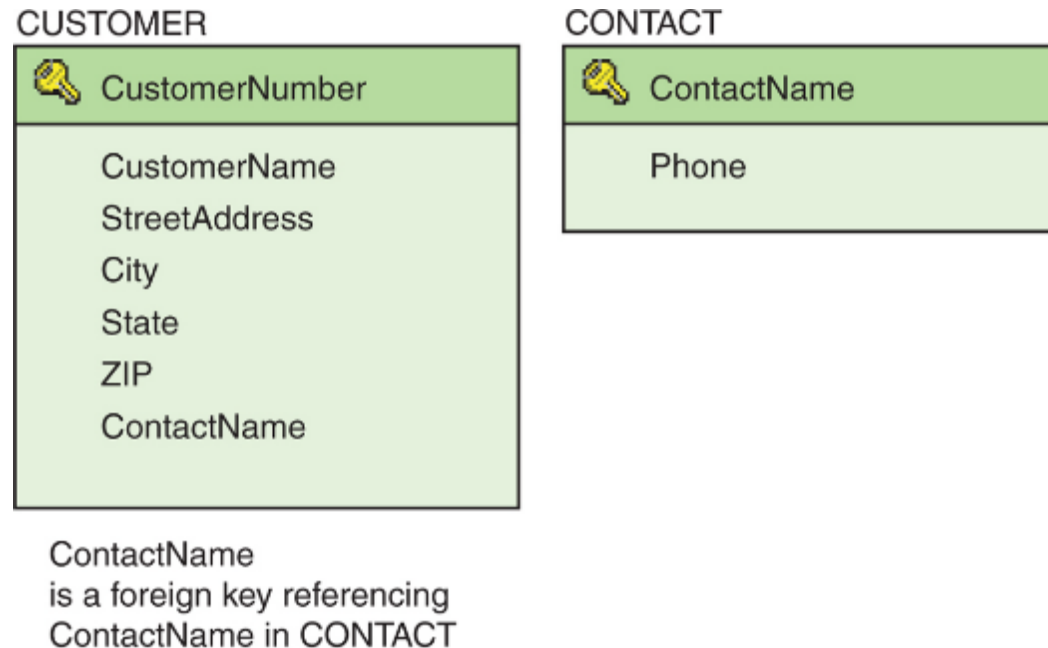


Denormalization

- Normalizing relations (or breaking them apart into many component relations) may significantly increase the complexity of the data structure.
- The question is one of balance.
 - Trading complexity for modification problems
- There are situations where denormalized relations are preferred.



The CUSTOMER Entity: The Denormalized Set of Tables



CUSTOMER (CustomerNumber, CustomerName, StreetAddress,
City, State, ZIP, *ContactName*)

CONTACT (ContactName, Phone)

Figure 5-6: The Denormalized CUSTOMER and Associated CONTACT Tables



Representing Weak Entities

- If not ID-dependent, use the same techniques as for strong entities.
- If ID-dependent, then must add primary key of the parent entity.

Representing Weak Entities Example

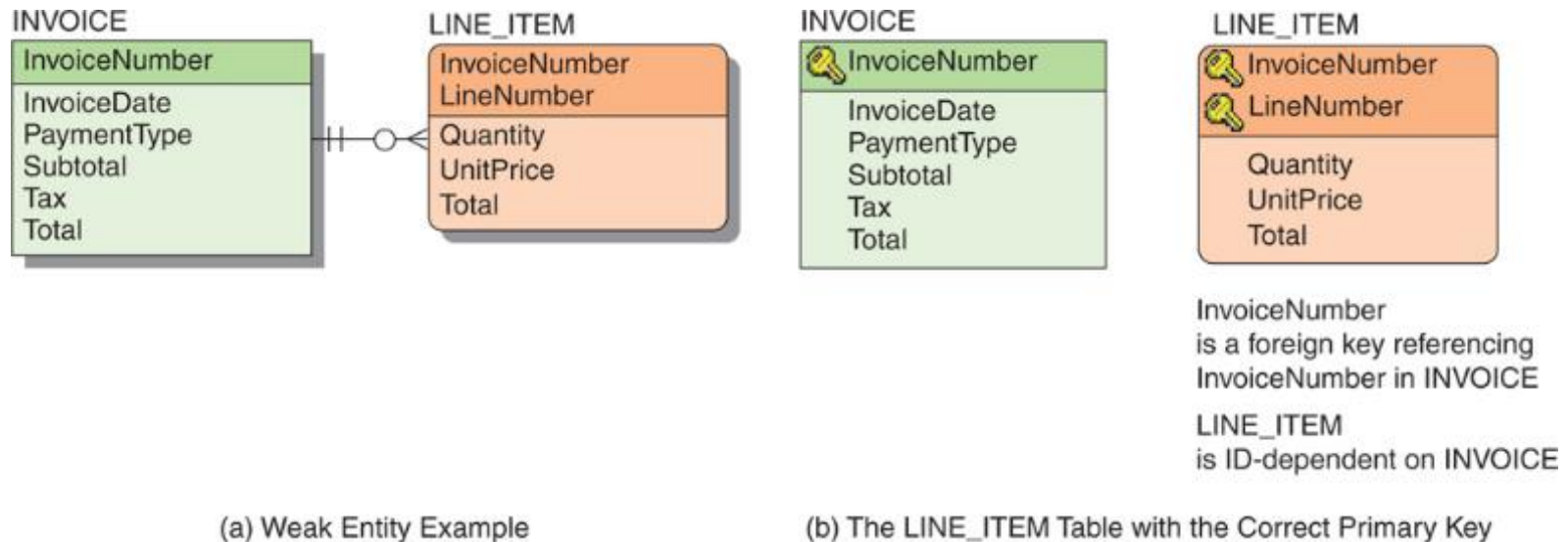


Figure 5-9: Relational Representation of a Weak Entity



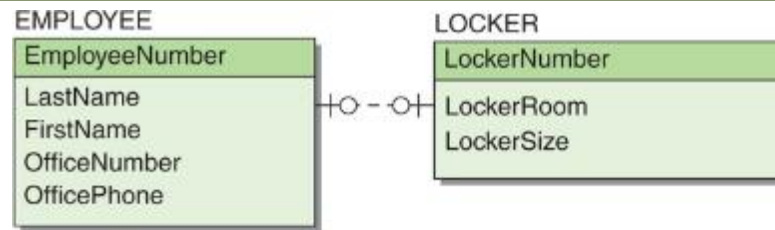
Representing Relationships

1:1 Relationships

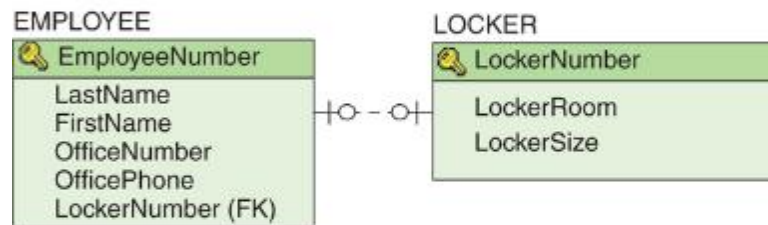
- The maximum cardinality determines how a relationship is represented.
- 1:1 relationship
 - The key from one relation is placed in the other as a *foreign key*.
 - It does not matter which table receives the foreign key.

Representing Relationships

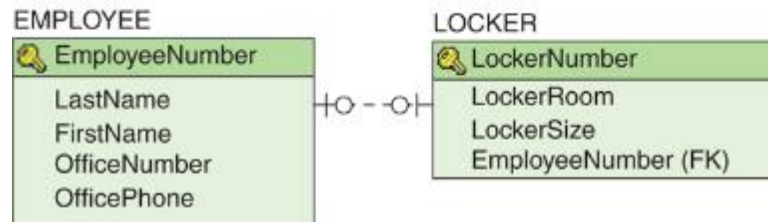
1:1 Relationship Example



(a) Example 1:1 Strong Entity Relationship



(b) Placing the Primary Key of LOCKER into EMPLOYEE



(c) Placing the Primary Key of EMPLOYEE into LOCKER

Figure 5-10: 1:1 Strong Entity Relationships



Representing Relationships SQL for 1:1 Relationships

```
SELECT      *  
FROM        LOCKER, EMPLOYEE  
WHERE       LOCKER.LockerNumber =  
            EMPLOYEE.LockerNumber ;
```

```
SELECT      *  
FROM        LOCKER, EMPLOYEE  
WHERE       LOCKER.EmployeeNumber =  
            EMPLOYEE.EmployeeNumber ;
```



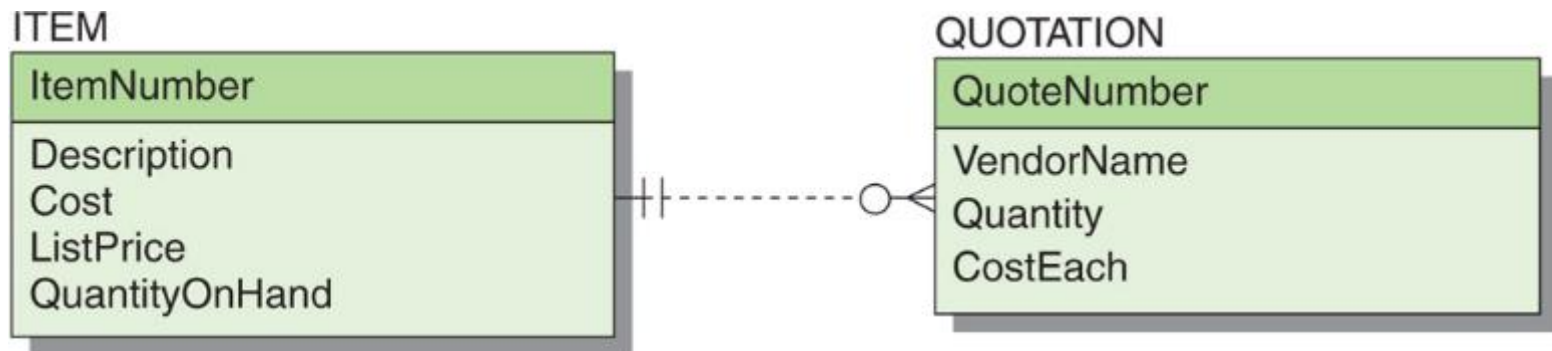
Representing Relationships

1:N Relationships

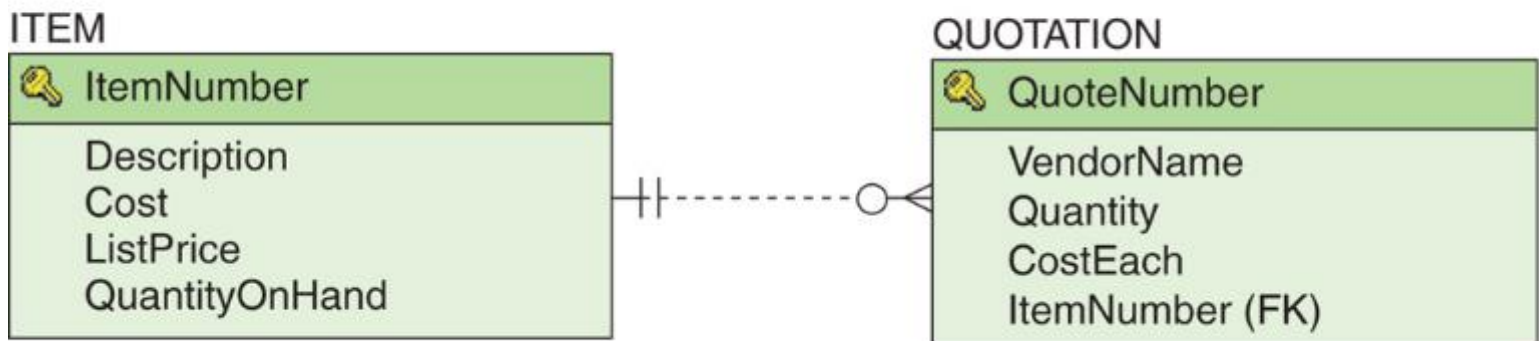
- Like a 1:1 relationship, a 1:N relationship is saved by placing the key from one table into another as a foreign key.
- However, in a 1:N the foreign key always goes into the many-side of the relationship.
 - The 1 side is called the **parent**.
 - The N side is called the **child**.

Representing Relationships

1:N Relationship Example



(a) 1:N Strong Entity Relationship Example

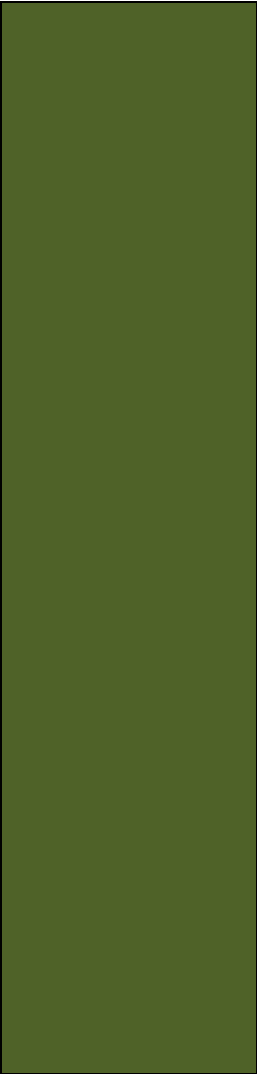


(b) Placing the Primary Key of ITEM into QUOTATION

Figure 5-12: 1:N Strong Entity Relationships



Representing Relationships SQL for 1:N Relationships



```
SELECT      *  
FROM        ITEM, QUOTATION  
WHERE       ITEM.ItemNumber =  
            QUOTATION.ItemNumber;
```



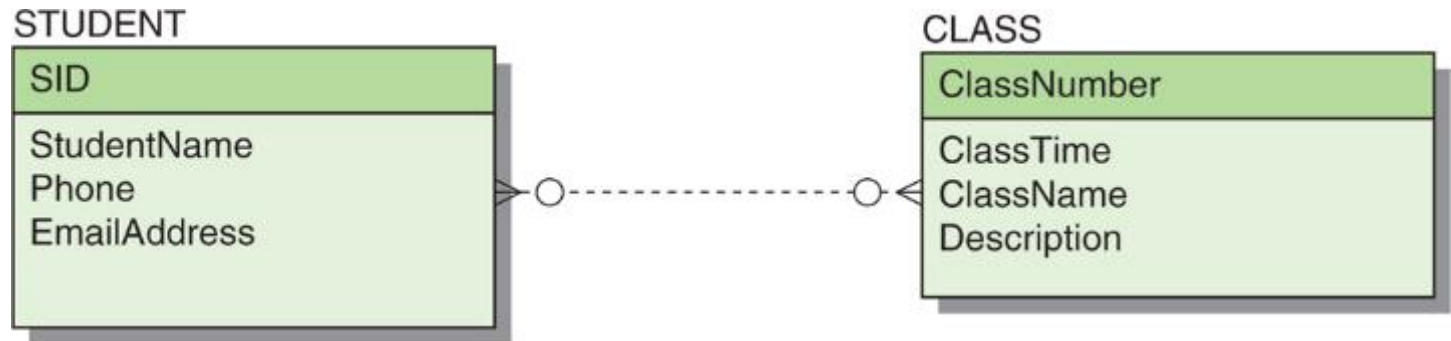
Representing Relationships

N:M Relationships

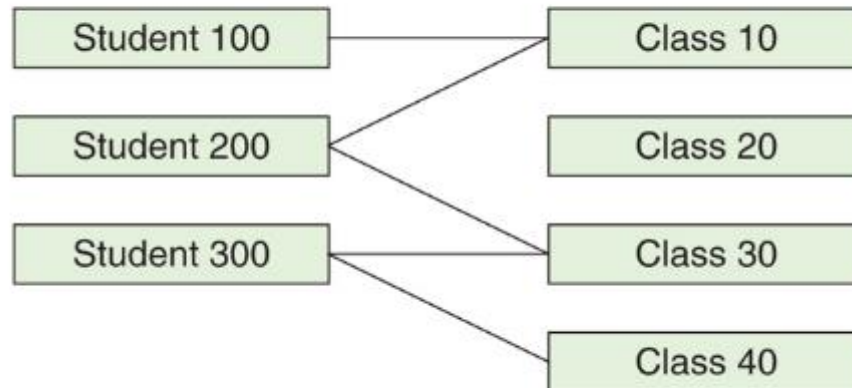
- To create an N:M relationship, a new table is created. This table is called an **intersection table**.
- An intersection table has a composite key consisting of the keys from each of the tables that it connects.

Representing Relationships

N:M Relationship – Data Model



(a) N:M Strong Entity Relationship Example

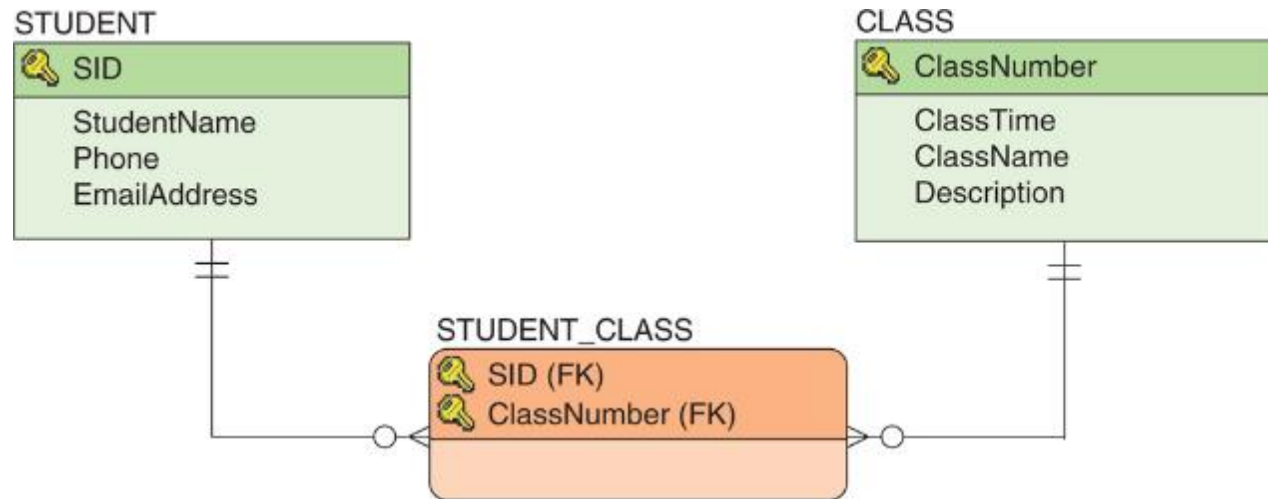


(b) Sample Data for the STUDENT-to-CLASS Relationship

Figure 5-13: N:M Strong Entity Relationships

Representing Relationships

N:M Relationship – Database Design



(a) The STUDENT_CLASS Intersection Table

100	Jones, Mary	100	10	10	Accounting
200	Parker, Fred	200	10	20	Finance
300	Wu, Jason	200	30	30	Marketing
		300	30	40	Database
		300	40		

(b) Sample Data for the STUDENT-to-CLASS Relationship

Figure 5-15: Representing an N:M Strong Entity Relationship



Representing Relationships

SQL for N:M Relationships

```
SELECT      *  
FROM        STUDENT, CLASS, STUDENT_CLASS  
WHERE       STUDENT.SID = STUDENT_CLASS.SID  
            AND  
            STUDENT_CLASS.ClassNumber =  
                CLASS.ClassNumber;
```

Representing Relationships

Association Relationships

- When an intersection table has columns beyond those in the primary key, the relationship is called an **association relationship**.

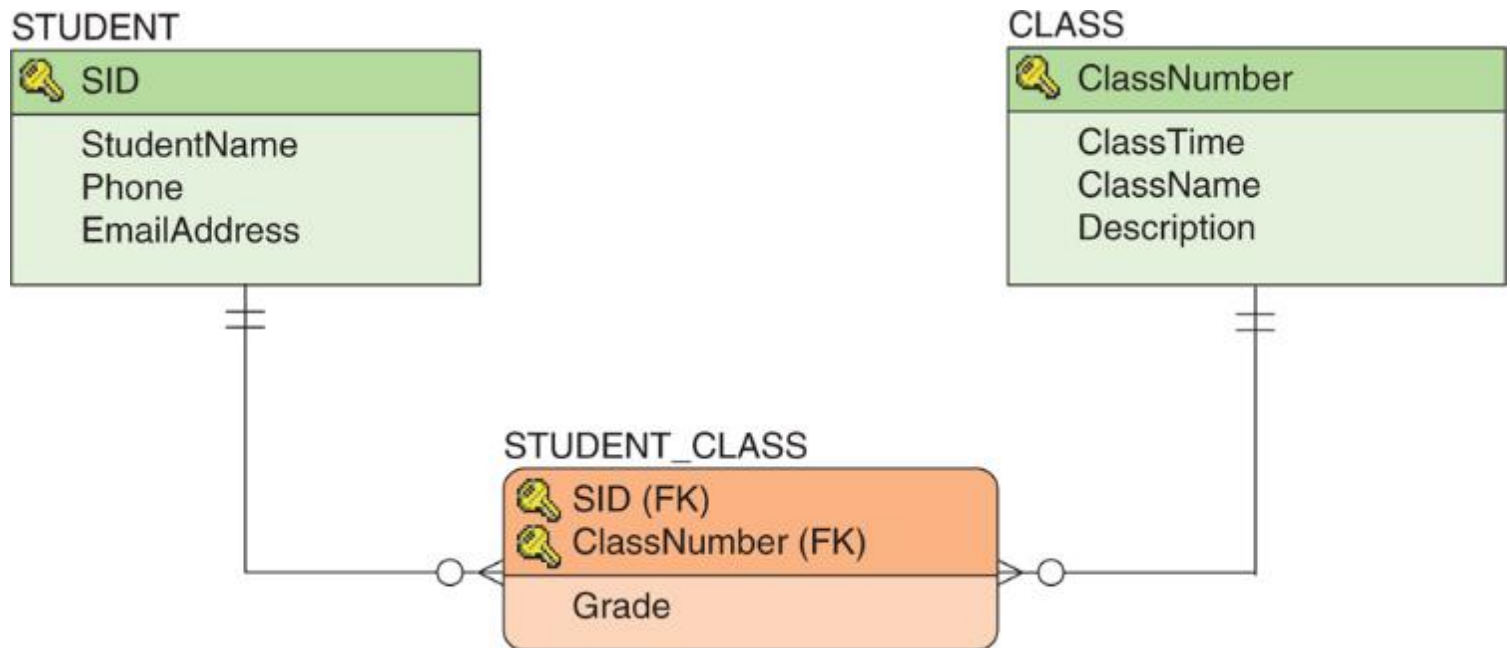


Figure 5-18: The Association Relationship

Representing Relationships

Supertype/Subtype Relationships

- The identifier of the supertype becomes the primary key and the foreign key of each subtype.

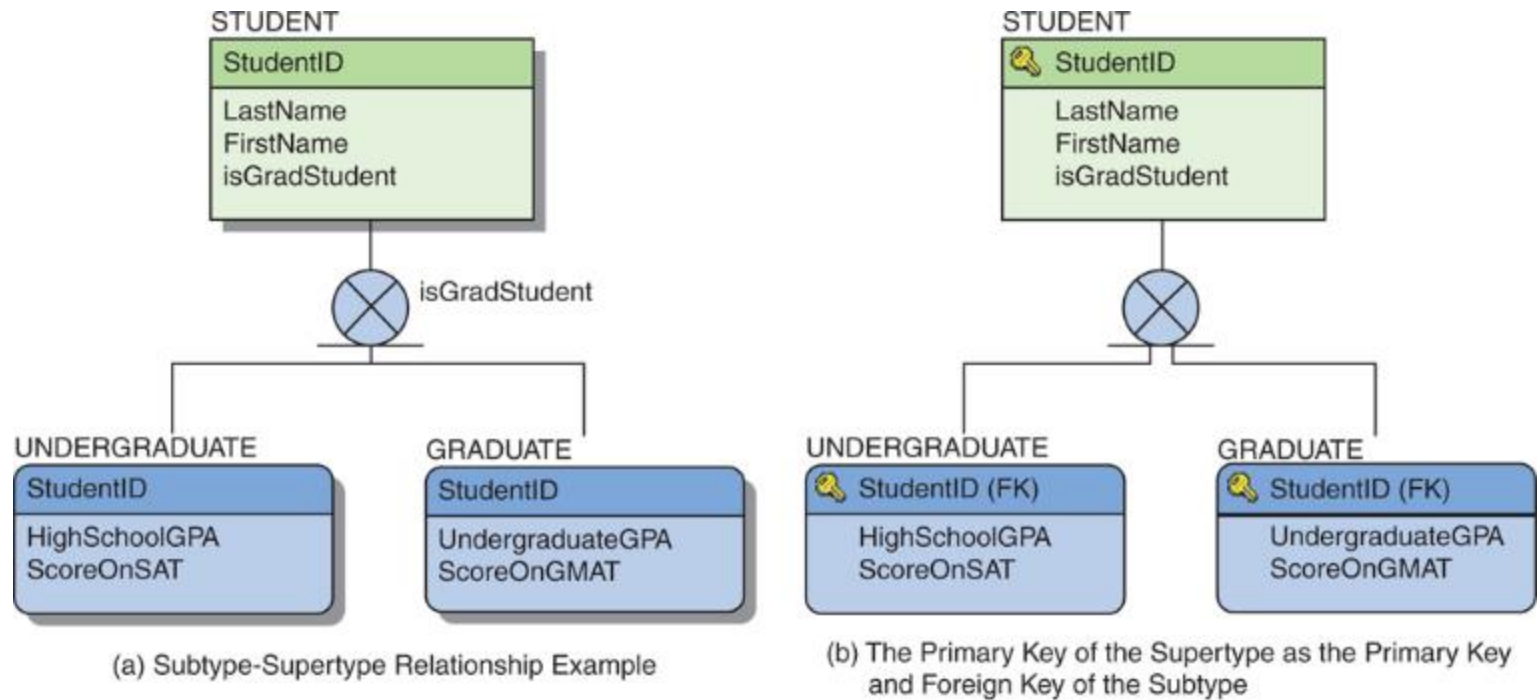


Figure 5-20: Representing Subtypes



Representing Relationships

Recursive Relationships

- A **recursive relationship** is a relationship that a relation has with itself.
- Recursive relationships adhere to the same rules as binary relationships.
 - 1:1 and 1:M relationships are saved using foreign keys.
 - M:N relationships are saved by creating an intersecting relation.

Representing Relationships

Recursive Relationships—Examples

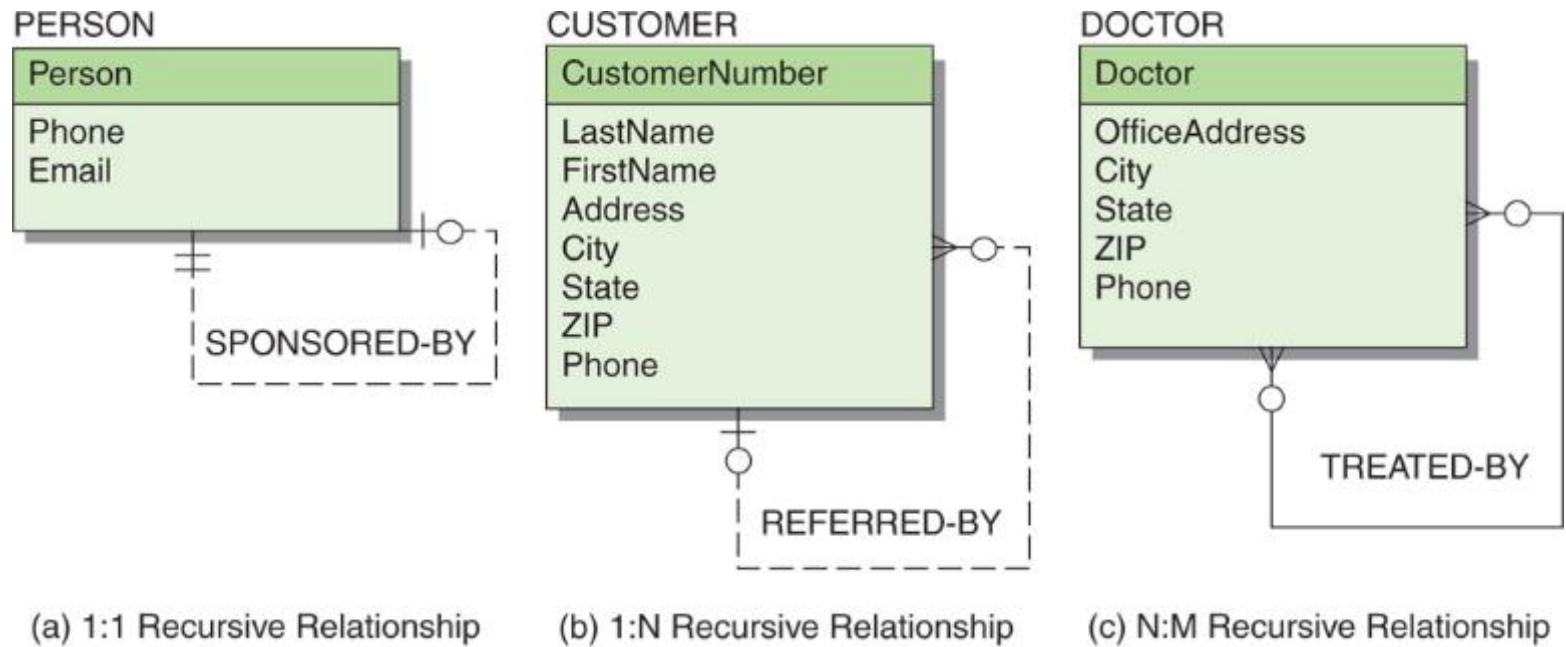
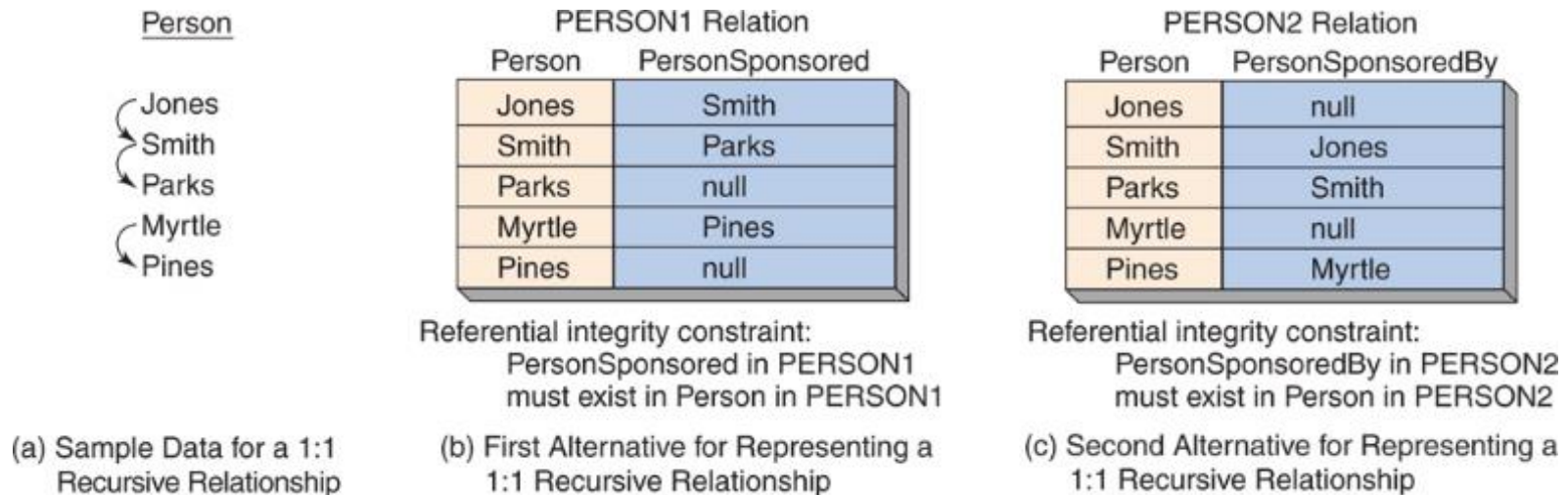


Figure 5-21: Example Recursive Relationships

Representing Relationships

1:1 Recursive Relationship Examples



```
SELECT *
FROM PERSON1 AS A, PERSON1 AS B
WHERE A.Person = B.PersonSponsored;
```

```
SELECT *
FROM PERSON2 AS C, PERSON2 AS D
WHERE C.Person = D.PersonSponsoredBy;
```

Figure 5-22: Example 1:1 Recursive Relationships

Representing Relationships

1:N Recursive Relationship Example

Customer Number Referred These Customers

100	200, 400
300	500
400	600, 700

(a) Sample Data for a 1:N Recursive Relationship

CUSTOMER Relation

CustomerNumber	CustomerData	ReferredBy
100	...	null
200	...	100
300	...	null
400	...	100
500	...	300
600	...	400
700	...	400

Referential integrity constraint:
ReferredBy in CUSTOMER must exist in
CustomerNumber in CUSTOMER

(b) Representing a 1:N Recursive Relationship
Within a Table

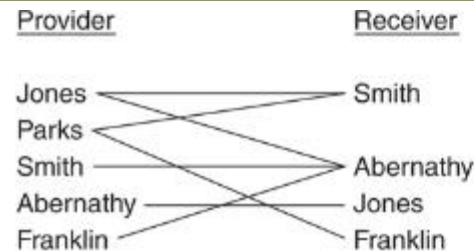
```

SELECT  *
FROM    CUSTOMER AS A, CUSTOMER AS B
WHERE   A.CustomerNumber = B.ReferredBy;
    
```

Figure 5-23: Example 1:N Recursive Relationship

Representing Relationships

N:M Recursive Relationship Example



DOCTOR Relation

Name	Other Attributes
Jones	...
Parks	...
Smith	...
Abernathy	...
O'Leary	...
Franklin	...

TREATMENT-INTERSECTION Relation

Physician	Patient
Jones	Smith
Parks	Smith
Smith	Abernathy
Abernathy	Jones
Parks	Franklin
Franklin	Abernathy
Jones	Abernathy

Referential integrity constraints:
 Physician in TREATMENT-INTERSECTION
 must exist in Name in DOCTOR

Patient in TREATMENT-INTERSECTION
 must exist in Name in DOCTOR

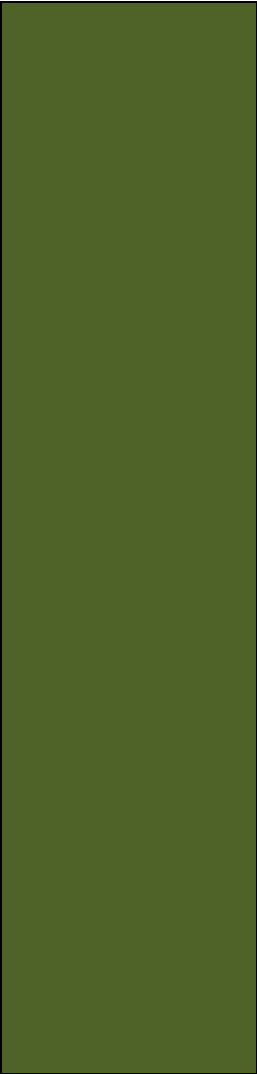
(a) Sample Data for an N:M Recursive Relationship

(b) Representing an N:M Recursive Relationship Using Tables

Figure 5-24: Example of an N:M Recursive Relationship



SQL for N:M Recursive Relationships



```
SELECT *  
FROM   DOCTOR AS A,  
        DOCTOR AS B,  
        TREATMENT-INTERSECTION  
WHERE  A.Name = TREATMENT-INTERSECTION.Physician  
        AND TREATMENT-INTERSECTION.Patient = B.Name;
```



Heather Sweeney Designs: Developing a Database Design

- Heather Sweeney Designs will be used as an ongoing example throughout Chapters 4, 5, 6, 7, and 8.
 - Heather Sweeney is an interior designer who specializes in home kitchen design.
 - She offers a variety of free seminars at home shows, kitchen and appliance stores, and other public locations.
 - She earns revenue by selling books and videos that instruct people on kitchen design.
 - She also offers custom-design consulting services.

Heather Sweeney Designs: Final Data Model

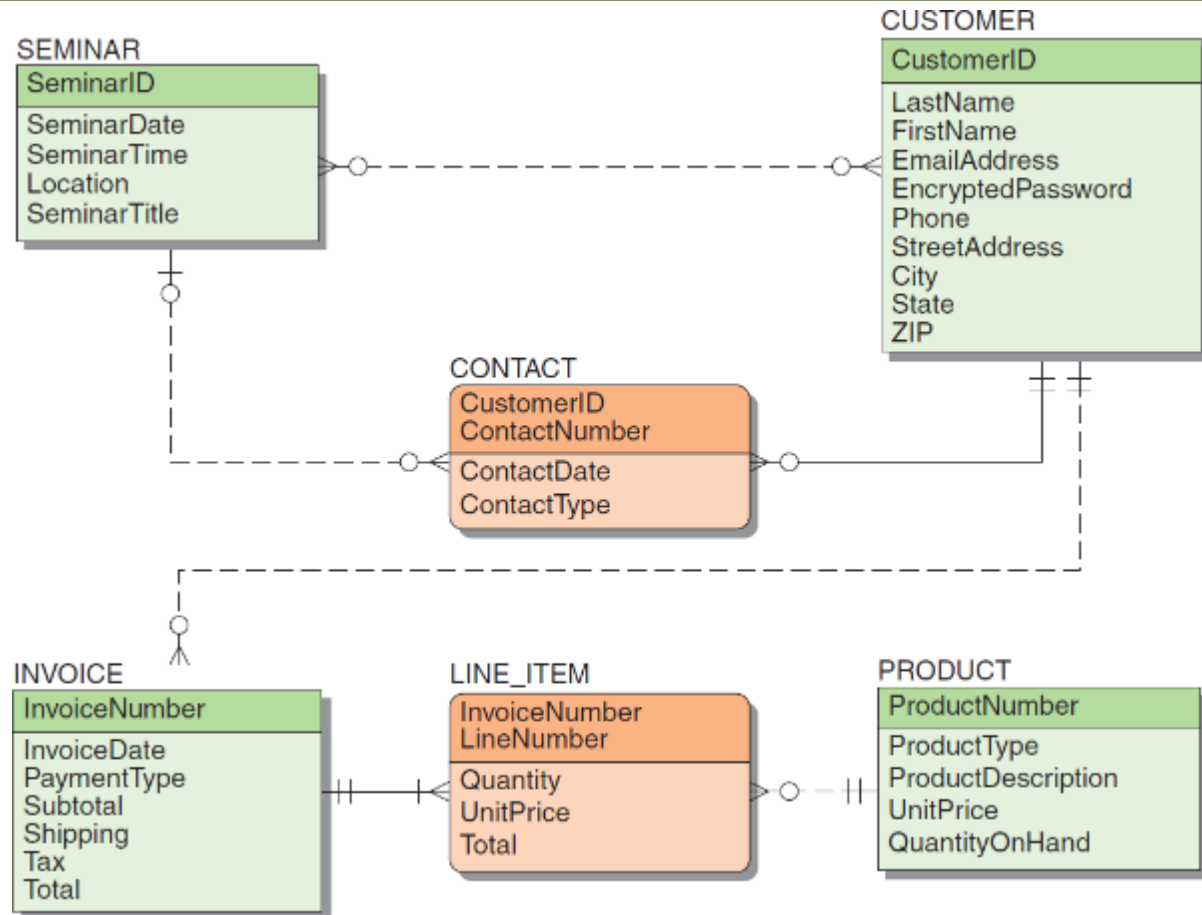


Figure 5-25: The Final Data Model for Heather Sweeney Designs



Specifying Column Properties

- Column properties must be specified for each table.
- The finalized column properties for the HSD tables are on the next set of slides-these are the column characteristics after additional needed foreign keys (shown in **bold**) have been added. This includes any new intersection tables.



HSD Column Property Specifications SEMINAR

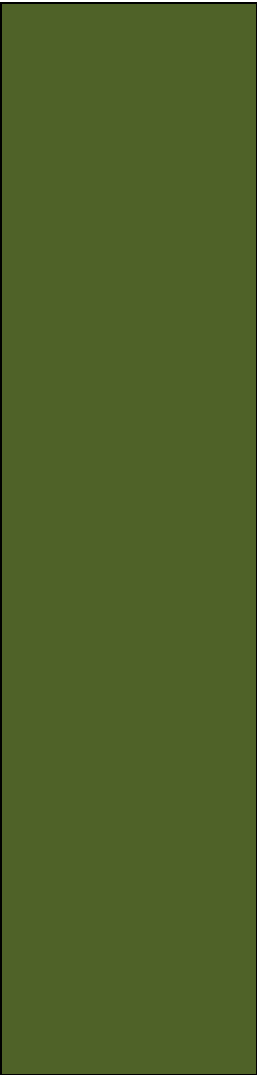
Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
SeminarID	Integer	Primary Key	Yes	DBMS supplied	Surrogate Key: Initial value=1 Increment=1
SeminarDate	Date	No	Yes	None	Format: yyyy-mm-dd
SeminarTime	Time	No	Yes	None	Format: 00:00:00.000
Location	VarChar (100)	No	Yes	None	
SeminarTitle	VarChar (100)	No	Yes	None	

SEMINAR

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications



HSD Column Property Specifications SEMINAR_CUSTOMER



Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
SeminarID	Integer	Primary Key, Foreign Key	Yes	None	
CustomerID	Integer	Primary Key, Foreign Key	Yes	None	

SEMINAR_CUSTOMER

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications

HSD Column Property Specifications

CUSTOMER

Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
CustomerID	Integer	Primary Key	Yes		DBMS Supplied Surrogate Key: Initial Value=1 Increment=1
LastName	Char (25)	No	Yes	None	
FirstName	Char (25)	No	Yes	None	
EmailAddress	VarChar (100)	Primary Key	Yes	None	
EncryptedPassword	VarChar(50)	No	No	None	
Phone	Char (12)	No	Yes	None	Format: ###-###-####
City	Char (35)	No	No	Dallas	
State	Char (2)	No	No	TX	Format: AA
ZIP	Char (10)	No	No	75201	Format: #####-####

CUSTOMER

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications

HSD Column Property Specifications

CONTACT

Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
CustomerID	Integer	Primary Key, Foreign Key	Yes	None	REF: CUSTOMER
ContactNumber	Integer	Primary Key	Yes	None	This is not quite a Surrogate Key—for <i>each</i> ContactNumber: Start=1 Increment=1 Application logic will be needed to supply the correct value
ContactDate	Date	No	Yes	None	Format: yyyy-mm-dd
ContactType	Char (15)	No	Yes	None	
SeminarID	Integer	Foreign Key	No	None	REF: SEMINAR

CONTACT

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications



HSD Column Property Specifications INVOICE

Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
InvoiceNumber	Integer	Primary Key	Yes	DBMS supplied	Surrogate Key: Initial value=35000 Increment=1
InvoiceDate	Date	No	Yes	None	Format: yyyy-mm-dd
CustomerID	Integer	Foreign Key	Yes	None	REF: CUSTOMER
PaymentType	Char (25)	No	Yes	Cash	
Subtotal	Numeric (9,2)	No	No	None	
Shipping	Numeric (9,2)	No	No	None	
Tax	Numeric (9,2)	No	No	None	
Total	Numeric (9,2)	No	No	None	

INVOICE

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications

HSD Column Property Specifications

LINE_ITEM

Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
InvoiceNumber	Integer	Primary Key, Foreign Key	Yes	None	REF: INVOICE
LineNumber	Integer	Primary Key	Yes	None	This is not quite a Surrogate Key—for <i>each</i> InvoiceNumber: Start=1 Increment=1 Application logic will be needed to supply the correct value
ProductNumber	Integer	Foreign Key	Yes	None	REF: PRODUCT
Quantity	Integer	No	No	None	
UnitPrice	Numeric (9,2)	No	No	None	
Total	Numeric (9,2)	No	No	None	

LINE_ITEM

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications



HSD Column Property Specifications PRODUCT

Column Name	Data Type (Length)	Key	Required	Default Value	Remarks
ProductNumber	Integer	Primary Key	Yes	DBMS supplied	Surrogate Key: Initial value=100 Increment=1
ProductType	Char(24)	No	Yes	None	
ProductDescription	VarChar (100)	No	Yes	None	
UnitPrice	Numeric (9, 2)	No	Yes	None	
QuantityOnHand	Integer	No	Yes	0	

PRODUCT

Figures 5-26 & 5-28: Heather Sweeney Designs Column Specifications

Heather Sweeney Designs: Database Design

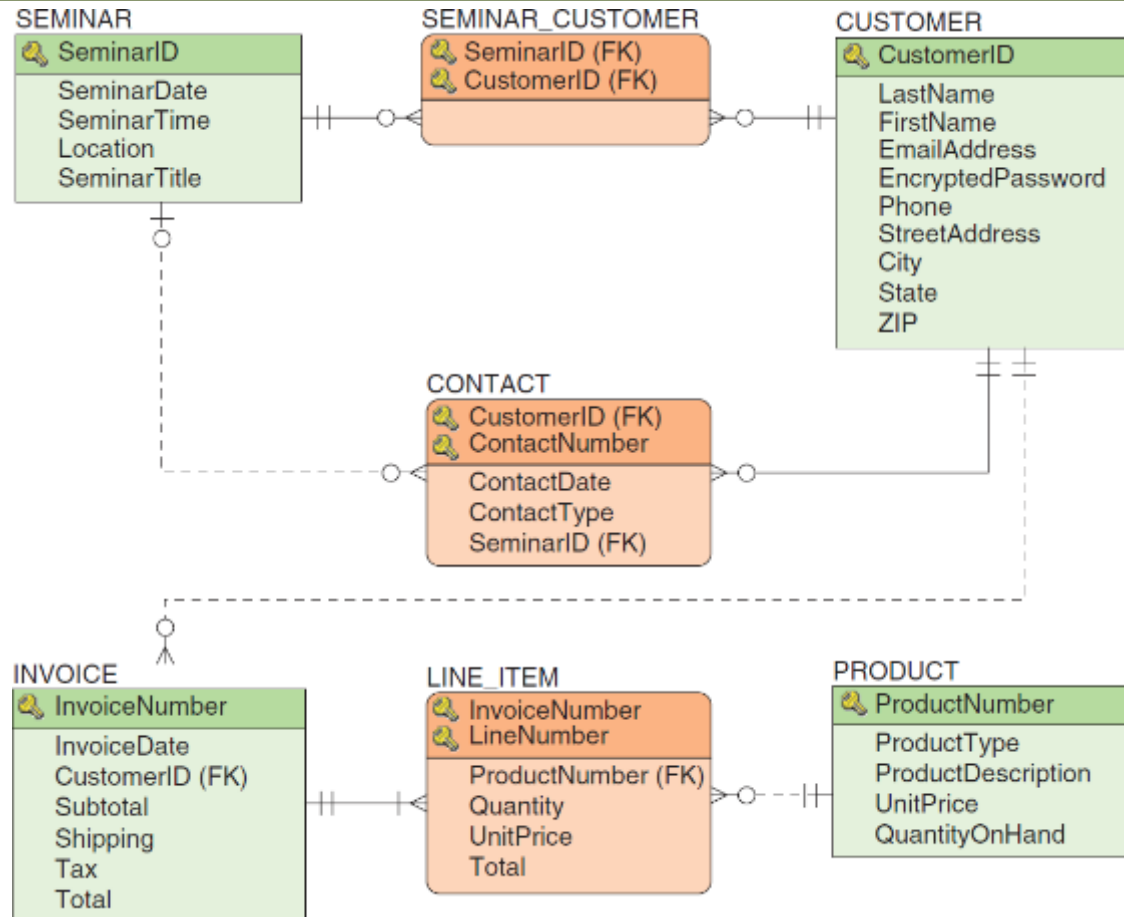


Figure 5-27: Database Design for Heather Sweeney Designs



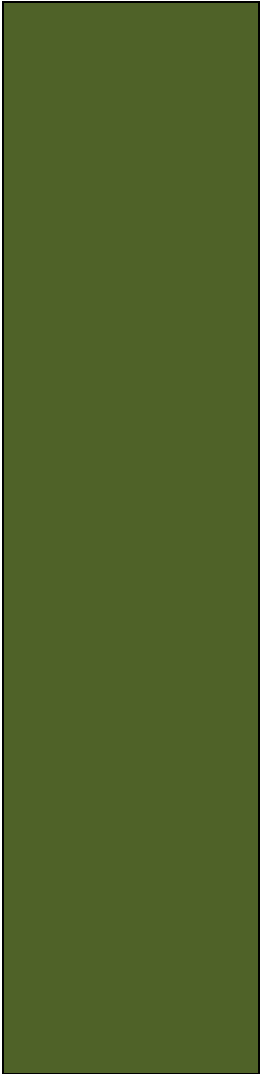
Heather Sweeney Designs: Database Design Schema

SEMINAR (SeminarID, SeminarDate, SeminarTime, Location, SeminarTitle)
CUSTOMER (CustomerID, LastName, FirstName, EmailAddress,
EncryptedPassword, Phone, StreetAddress, City, State, ZIP)
SEMINAR_CUSTOMER (SeminarID, CustomerID)
CONTACT (CustomerID, ContactDate, ContactDate, ContactType, *SeminarID*)
PRODUCT (ProductNumber, ProductType, ProductDescription, UnitPrice,
QuantityOnHand)
INVOICE (InvoiceNumber, InvoiceDate, *CustomerID*, PaymentType, SubTotal, Tax,
Total)
LINE_ITEM (*InvoiceNumber*, LineNumber, *ProductNumber*, Quantity, UnitPrice,
Total)

Heather Sweeney Designs: Referential Integrity Constraints

Relationship		Referential Integrity Constraint	Cascading Behavior	
Parent	Child		On Update	On Delete
SEMINAR	SEMINAR_CUSTOMER	SeminarID in SEMINAR_CUSTOMER must exist in SeminarID in SEMINAR	No	No
CUSTOMER	SEMINAR_CUSTOMER	CustomerID in SEMINAR_CUSTOMER must exist in CustomerID in CUSTOMER	No	No
SEMINAR	CONTACT	SeminarID in CONTACT must exist in SeminarID in SEMINAR	No	No
CUSTOMER	CONTACT	CustomerID in CONTACT must exist in CustomerID in CUSTOMER	No	No
CUSTOMER	INVOICE	CustomerID in INVOICE must exist in CustomerID in CUSTOMER	No	No
INVOICE	LINE_ITEM	InvoiceNumber in LINE_ITEM must exist in InvoiceNumber in INVOICE	No	Yes
PRODUCT	LINE_ITEM	ProductNumber in LINE_ITEM must exist in ProductNumber in PRODUCT	Yes	No

Figure 5-29: Referential Integrity Constraint Enforcement for Heather Sweeney Designs



END